



DAY 5 • PROJECT 1

AI Pipelines for Document Analysis

From Text to Structured Insight

Building the Resume × Job Description Analyzer

By the End of This Morning

- 1 Explain why one prompt can't do a recruiter's job — and design a pipeline that can.
- 2 Turn a résumé and a job post into structured JSON your code can trust.
- 3 Score a match the way an ATS does — and explain exactly why.

This is not prompt engineering, round two. It's software engineering — applied to prompts.

This Morning

1

One Prompt Isn't Enough

Why chatbots fail at analysis

2

The Assembly Line

Parse → Extract → Analyse → Score

3

Unstructured → Structured

JSON as a contract between stages

4

Prompts That Judge, Not Just Chat

Extraction vs. evaluation vs. feedback

5

Trust the Number

Scoring, thresholds, and when the model misbehaves

SECTION 01 OF 05

One Prompt Isn't Enough

Why asking a chatbot to extract, evaluate, score, and advise all at once backfires.

01

The Recruiter's Afternoon

200

résumés, one afternoon

"Read this résumé, decide if the skills match, give me a percentage, and explain what's missing — all in one response."



Inconsistent format

A different shape every time — nothing to compare across candidates.



Possible hallucination

The model may invent a skill the applicant never listed.



Not comparable

Free-form prose can't be ranked, filtered, or stored.

Three Reasons One Call Fails

1

Attention dilution

Multiple subtasks at once means less quality on each one.

2

Format conflict

Tight JSON and flowing prose fight inside one response.

3

No intermediate check

A bad extraction poisons everything built on top of it.

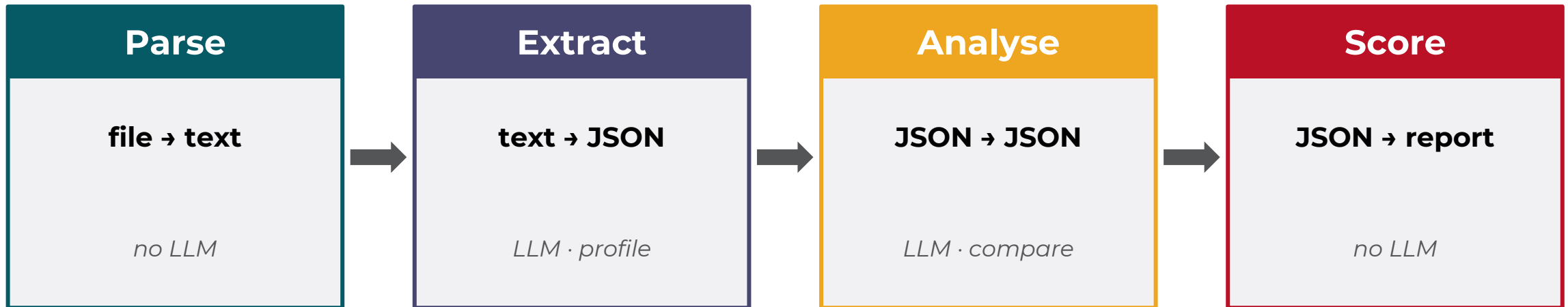
SECTION 02 OF 05

The Assembly Line

Each station does exactly one job — and you can test it without the others existing.

02

One Job Per Station



Data flows left to right. No stage looks back — and none knows what any other stage does internally.

Every Step Reduces Ambiguity

Stage Boundary	Before	After
File → Text	resume.pdf (binary bytes)	"Aiko Tanaka ... Skills: Python, AWS"
Text → JSON	"...proficient in Python, cloud projects..."	{ skills: { languages: [Python] } }
JSON → Analysis	profile JSON + requirements JSON	{ present, missing, score: 72 }
Analysis → Report	{ score: 72, missing: [Docker] }	"ATS score 72/100. Bonus: Docker."

SECTION 03 OF 05

Unstructured → Structured

Prose says a lot; a computer can't act on it. JSON is the bridge — and the contract.

03

The LLM Is a Translator

UNSTRUCTURED TEXT

Experienced in Python and cloud infrastructure. Led a team of 3. Familiar with AWS, Docker, and some Kubernetes.

LLM

STRUCTURED JSON

```
{  
  "languages": ["Python"],  
  "tools": ["AWS", "Docker"],  
  "leadership": true  
}
```

A dict lookup can't read prose. An LLM reads prose and writes dictionaries — that's the bridge.

The Schema Is a Contract

Rename one field `"missing"` → `"gaps"` in Stage 3.

Stage 4 reads it with `result.get("missing", [])`.

No KeyError. No crash. Just a report that silently claims nothing is missing — when something is.

Schema-first design: decide the exact JSON shape before writing the prompt. In ICCO terms, the schema *is* the Output component.

SECTION 04 OF 05

Prompts That Judge

Not just chat: extraction copies facts, evaluation judges them, feedback explains them. Never confuse the three.

0

4

Three Prompts, Three Jobs

Extraction

Copy facts from text into a schema.

temp 0.0 – 0.1

Only extract. Never invent.

Evaluation

Compare two structured objects against a rubric.

temp 0.1 – 0.3

Cite the rubric. Score strictly.

Feedback

Translate results into actionable advice.

temp 0.3 – 0.6

Feedback only. Never rewrite.

Temperature: A Precision Dial

Fully deterministic → more nuance, still parseable → unreliable for JSON



Above 0.7, JSON-returning prompts risk malformed structure. Extraction stays near zero — it's copying, not creating.

SECTION 05 OF 05

Trust the Number

A composite score only means something next to a threshold — and a way to explain it.



Weighted Aggregation

Dimension	Weight	Score
Keyword match	0.5	80
Bullet quality	0.3	65
Structure audit	0.2	70

$$S = \sum w_i \cdot s_i \quad (\text{weights sum to 1.0})$$

73.5

out of 100 — composite score

*Weights encode the evaluator's priorities.
Choose them deliberately — and document
them.*

The Threshold Makes the Score Mean Something

```
{  
  "composite_score": 64,  
  "pass_threshold": 60,  
  "result": "PASS",  
  "margin": 4  
}
```



64 clears a **60** threshold by a margin of **+4**. Against a 70% bar, the same résumé fails.

Singapore ATS thresholds typically sit at 60–70% for entry-level tech roles — a heuristic, not a law. Report the score, the threshold, and the margin together.

The LLM Is Not a Database

Leading prose

"Here is the JSON you requested:"

Markdown fences

```
```json ... ``` wrapped around the object
```

## Wrong types

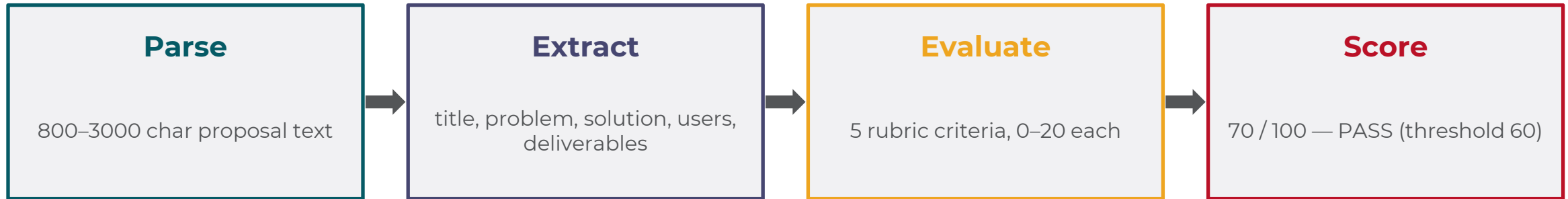
An array field returns as a bare "string"

## Truncation

Response hits max\_tokens mid-object

*Never return None or {} on a parse failure. Raise — and let the caller retry.*

# Worked Example: The Proposal Analyser



```
"user_specificity": { "score": 10, "note": "Target users described only as 'students' – too broad." }
```

*An observation the student can act on — not a rewritten user-persona paragraph handed to them.*

## CONTRIBUTIONS

# What You Can Now Do

- ✓ Explain why pipelines beat a single prompt — and design one that separates parse, extract, analyse, and score.
- ✓ Write a schema-first extraction prompt and a rubric-grounded evaluation prompt, at the right temperature.
- ✓ Compute a weighted, threshold-checked composite score — and defend every number in it.

---

**This afternoon:** build the Resume × Job Description Analyzer.

**Questions?**